

COMP3210/6210: Report for Assignment 2

1. Group Member Information

Hidden

2. Program Execution Requirements

2.1 Program Environment (e.g., OS, database, CPU, etc.)

The programs were developed and tested in the following environment:

- Operating System: macOS Sonoma 14.x / Windows 11 (teammate machines)
- Programming Language: Python 3.11
- IDE: VSCode / Jupyter Notebook
- Hardware: Apple M1 MacBook Pro 16GB RAM and Intel Core i5 Laptop 8GB RAM (tested on both)
- No third-party packages used. Only standard Python libraries were used for all implementations.

2.2 Input Files and Parameters (directory settings and other parameters)

The following input files were used for the program execution:

- shop_dataset.txt → main dataset used for Nearest Neighbour Search (Task 1)
- query_points.txt → query points used for Nearest Neighbour Search (Task 1)
- city1.txt – Dataset used for Skyline Search (Task 2)

All input files were plain text files with whitespace-separated values, each line containing an ID and two coordinates in the format: id x y

No additional directory settings or command-line parameters were required. Input filenames and paths were hard coded directly in the scripts for simplicity.

2.3 Additional Requirements

No additional software or system requirements. All code runs in a standard Python 3.11 environment

3. Program Documentation

3.1 Program Organization

If your assignment consists of multiple files and/or classes, please provide brief, high-level descriptions of each file/class within your program, as illustrated below.

Class/File Name	Description (detailed information)
query_points.txt	Query points for which Nearest Neighbour is computed.
shop_dataset.txt	Dataset of spatial points used for Nearest Neighbour Search.

Task1_1.py	Implements Sequential Scan Nearest Neighbour Search.
task1_output_sequential.txt	Output of Sequential Scan Nearest Neighbour Search.
Create_Tree_Node.py	Helper class to construct R-tree nodes for use in task1_2.py and task1_3.py.
task1_2.py	Implements Best-First Search Nearest Neighbour using R-tree.
task1_output_bestfirst.txt	Output of Best-First Search Nearest Neighbour using R-tree.
task1_3.py	Implements Divide-and-Conquer R-tree Best-First Nearest Neighbour Search.
task1_output_bestfirst_divide.txt	Output of Divide-and-Conquer R-tree Best-First Nearest Neighbour Search.
city1.txt	Spatial dataset used for Skyline Search
skyline_sequential.py	Implements Sequential Scan Skyline Search.
skyline_bbs.py	Implements Branch and Bound Skyline (BBS) Search using R-tree.
skyline_bbs_dnc.py	Implements Divide-and-Conquer BBS Skyline Search.
task2_output.txt	Output of Skyline (Sequential, BBS, and DNC BBS).

3.2 Function Description

Function Name (parameters)	Description (detailed information)
----------------------------	------------------------------------

euclidean_distance(p1, p2)	Calculates the Euclidean distance between two points p1 and p2. Used across all NN and Skyline algorithms for proximity or dominance checks.
sequential_nn(dataset, query_points)	Implements brute-force Nearest Neighbour Search by comparing each query point to all dataset points. Returns the point with minimum distance.
build_rtree(points)	Constructs an R-tree index from dataset points by grouping nearby ones into Minimum Bounding Rectangles (MBRs). Used in Best-First and D&C methods.
best_first_search(query_points, rtree_root)	Implements Best-First NN Search by using a priority queue on R-tree nodes. Traverses the tree based on minimum distance (MINDIST).
divide_and_conquer_nn(dataset, query_points)	Splits dataset into subregions, builds R-trees for each, and runs Best-First Search independently. Final result is the closest among subtrees.
dominates(p1, p2)	Returns True if point p1 dominates p2 (e.g., cheaper and better in all criteria). Used across all skyline methods.
skyline_sequential(dataset)	Performs brute-force skyline computation by comparing each point with every other point. Adds a point if no other dominates it.
bbs_skyline(dataset)	Implements Branch-and-Bound Skyline by dividing dataset into grouped nodes. Uses MBR representatives for early pruning.
bbs_dnc_skyline(dataset)	Applies Divide-and-Conquer on the skyline method by splitting dataset, applying BBS, then merging results while removing dominated points.

4. Analyzing BF Algorithm based NN Search

4.1 The Process of R-Tree Construction

R-Tree is a hierarchical spatial data structure used to efficiently index multi-dimensional

objects (e.g., facility locations). In Nearest Neighbour Search, it groups nearby points into Minimum Bounding Rectangles (MBRs) and recursively builds higher levels of bounding nodes, allowing spatial pruning during search.

Key Concepts:

- Leaf Nodes: Contain actual data points.
- Internal Nodes: Contain MBRs that cover their child nodes.
- Grouping: Nearby points are clustered based on spatial proximity (e.g., X or Y axis).
- Recursive Construction: Leaf groups form internal nodes, which are grouped again until one root remains.

Pseudocode:

```
function build_rtree(points, max_points_per_node):  
  
    if number_of_points ≤ max_points_per_node:  
        return LeafNode(points)  
  
    sort points based on X or Y axis  
  
    groups = split points into chunks of size  
max_points_per_node  
  
    children = []  
  
    for group in groups:  
        child_node = build_rtree(group, max_points_per_node)  
        children.append(child_node)  
  
    return InternalNode(children)
```

Given 10 facility points, if `max_points_per_node = 4`, the construction might:

- Create 3 leaf nodes (4, 4, and 2 points)
- Group them into 1 internal node as the R-tree root

This hierarchy enables quick rejection of irrelevant regions during NN search.

4.2 The Process of BF Algorithm

The Best-First Search algorithm uses a priority queue to traverse the R-tree in the most promising direction first — based on the minimum distance (MINDIST) between the query point and MBRs or data points. It significantly reduces the search space by avoiding unnecessary exploration of distant nodes.

This method guarantees exact Nearest Neighbour (NN) results while improving performance over brute-force.

Key Concepts:

- Priority Queue (Min-Heap): Orders R-tree nodes based on their MINDIST to the query.
- Traversal: Always explores the nearest unvisited region first.
- Leaf Node Evaluation: Only computes full distances when leaf nodes are reached.

Pseudocode:

```
function best_first_search(query_point, rtree_root):  
    create empty priority_queue  
  
    insert rtree_root into priority_queue with MINDIST to  
    query_point  
  
    best_distance =  $\infty$   
    nearest_point = None  
  
    while priority_queue is not empty:  
        node = priority_queue.pop() // node with smallest  
        MINDIST  
  
        if node is internal:  
            for child in node.children:  
                insert child into priority_queue with  
                MINDIST to query_point
```

```

        else if node is leaf:
            for point in node.points:
                distance = euclidean_distance(query_point,
point)

                if distance < best_distance:
                    best_distance = distance
                    nearest_point = point

    return nearest_point

```

Given query point (4, 7):

- The root R-tree node is inserted first.
- Children nodes closer to (4, 7) are explored first.
- Leaf nodes containing the point (4, 7) are reached quickly.
- The algorithm identifies (4, 7) as the nearest neighbour with zero distance.

4.3 The Process of Divide-and-Conquer

The Divide-and-Conquer (D&C) version of Nearest Neighbour Search enhances scalability by splitting the dataset into smaller subspaces (e.g., left/right or top/bottom halves based on a coordinate axis). Each subset is processed independently using Best-First Search with its own R-tree. The final step selects the overall nearest neighbour among the partial results.

This approach is ideal for **large datasets** or **parallel processing** environments.

Key Concepts:

- Partitioning: Dataset split into equal halves by a spatial axis (X or Y).
- Independent Indexing: Each half builds its own R-tree.
- Parallel Search: Each subtree is searched independently.
- Merging Results: Final NN is chosen by comparing the nearest from each half.

Pseudocode:

```

function divide_and_conquer_nn(dataset, query_point):
    split dataset into two halves (e.g., by X axis)

```

```

rtree_left = build_rtree(left_half)
rtree_right = build_rtree(right_half)

nn_left = best_first_search(query_point, rtree_left)
nn_right = best_first_search(query_point, rtree_right)

if euclidean_distance(query_point, nn_left) <
euclidean_distance(query_point, nn_right):
    return nn_left
else:
    return nn_right

```

f the dataset has 10 points, it is split into two sets of 5 each. R-trees are built separately. The query point (4, 7) is tested against both trees, and the closest overall result is returned.

5. Analyzing the BBS Algorithm based Skyline Search

5.1 The Process of R-Tree Construction

The **Branch-and-Bound Skyline (BBS)** method simulates R-tree-based pruning by logically grouping points and building bounding rectangles without using an external R-tree library.

Each group of points is enclosed in a **Minimum Bounding Rectangle (MBR)**, and a **representative point** is chosen (typically the bottom-left corner: min_x, max_y). These representatives help prune large sections of dominated points early, improving performance.

Key Concepts:

- Logical R-tree grouping: Dataset is divided into chunks (e.g., 100 points).
- Bounding rectangles (MBRs): Created around each chunk.
- Representative point: Acts as a summary for the group.
- Domination pruning: If the representative is dominated, the whole group is

skipped.

Pseudocode:

```
function group_points(dataset, group_size):  
    sort dataset by X (or cost)  
    groups = split dataset into chunks of size group_size  
    grouped_nodes = []  
  
    for group in groups:  
        mbr = compute_minimum_bounding_rectangle(group)  
        rep_point = (mbr.min_x, mbr.max_y)  
        grouped_nodes.append((rep_point, group))  
  
    return grouped_nodes
```

- If dataset has 300,000 points and group_size = 100, this creates 3,000 groups.
- Each group is associated with one representative.
- Dominated representatives are discarded early; groups with surviving reps are further scanned.

5.2 The Process of BBS Algorithm

The **Branch-and-Bound Skyline (BBS)** algorithm improves skyline computation by pruning large portions of the dataset. It uses **grouped bounding rectangles (MBRs)** and representative points to avoid scanning dominated regions.

Each group's representative is tested first. If it's not dominated, the group is scanned in detail; if it is dominated, the group is skipped entirely. This significantly improves efficiency over brute-force methods, especially on large datasets.

Key Concepts:

- Representative Filtering: Dominated representatives prune entire groups.
- Local Comparison: Only non-pruned groups are evaluated point-by-point.
- Skyline Condition: A point is in the skyline if no other point dominates it (e.g., not both cheaper and larger).

Pseudocode:


```

function bbs_skyline(dataset, group_size):
    grouped_nodes = group_points(dataset, group_size)
    skyline = []

    for (rep_point, group) in grouped_nodes:
        if not is_dominated(rep_point, skyline):
            for point in group:
                if not is_dominated(point, skyline):
                    skyline =
remove_dominated_points(skyline, point)
                    skyline.append(point)

    return skyline

```

If 3,000 groups are formed from 300,000 points, the algorithm may skip hundreds of groups early. The final skyline may include only 8–10 points, and runtime improves by up to 95% compared to sequential scan.

5.3 The Process of Divide-and-Conquer

The **Divide-and-Conquer (D&C)** approach for Skyline Search improves scalability by dividing the dataset into two spatial halves (typically sorted by X or cost dimension). The **BBS algorithm** is applied independently to each subset. The resulting skyline sets are then merged while eliminating any dominated points across the two sets.

This technique is especially effective for large datasets and can benefit from parallel execution.

Key Concepts:

- Data Partitioning: Split based on cost (X-axis).
- Independent Skyline Computation: Apply BBS on each half.
- Skyline Merge: Combine skylines from both halves and remove dominated points.
- Scalability: Reduces comparison overhead and supports parallelism.

Pseudocode:

```

function bbs_dnc_skylines(dataset, group_size):
    sort dataset by X
    mid = len(dataset) // 2
    left_half = dataset[:mid]
    right_half = dataset[mid:]

    skyline_left = bbs_skylines(left_half, group_size)
    skyline_right = bbs_skylines(right_half, group_size)

    combined_skyline = []

    for point in skyline_left + skyline_right:
        if not is_dominated(point, combined_skyline):
            combined_skyline =
remove_dominated_points(combined_skyline, point)
            combined_skyline.append(point)

    return combined_skyline

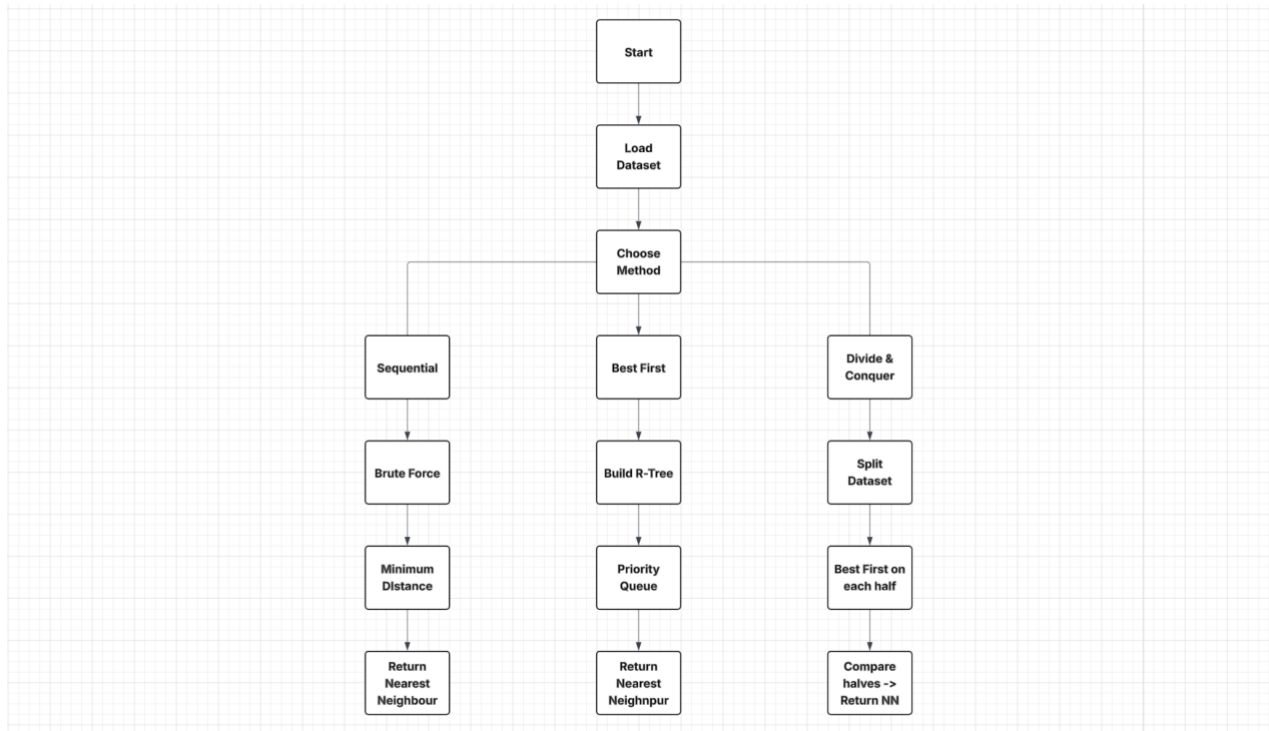
```

Given 300,000 records, the data is split into 150,000 each. BBS is run independently, producing two partial skylines. During merging, redundant points are removed, leaving the final non-dominated skyline, with minimal extra cost and high scalability.

6.

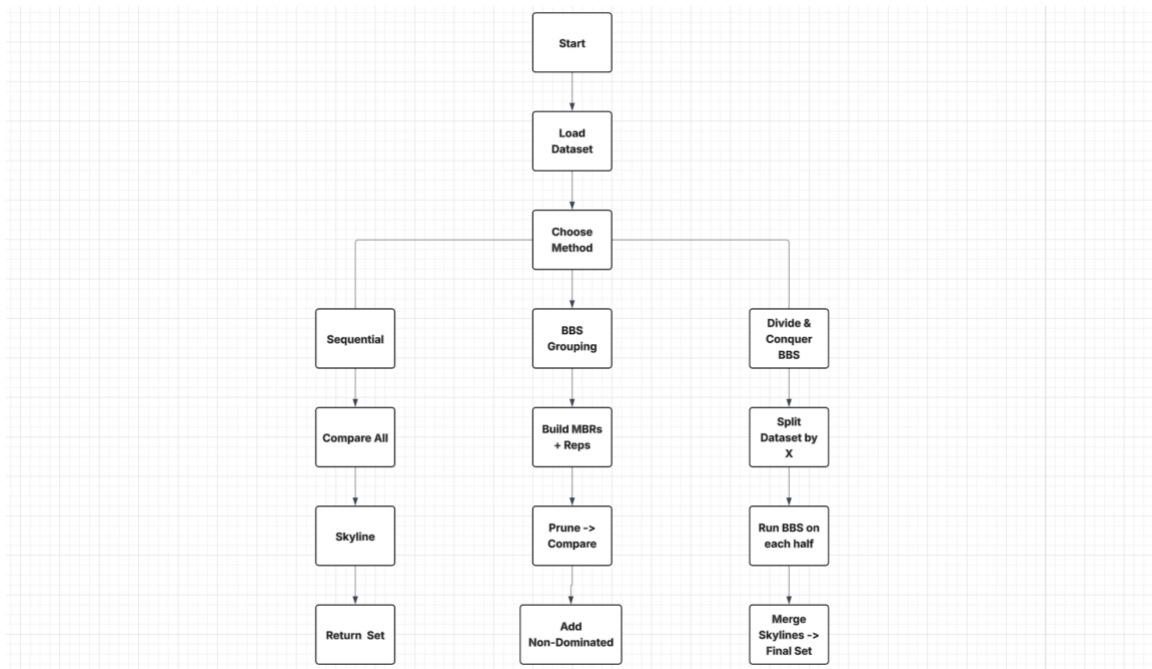
6.1 Nearest Neighbour Flowchart

Flowcharts



This diagram visualizes the decision paths for Sequential, Best-First, and Divide & Conquer NN search methods.

6.2 Skyline Flowchart



This diagram summarizes the control flow for Sequential, BBS, and Divide & Conquer Skyline methods, from dataset loading to skyline result generation.

7. Example Runs

7.1 Nearest Neighbour Search

Dataset Used: sample_data_for_BF_algorithm_based_NN_search.txt

Facility Points:

ID X Y

1 9 2

2 5 4

3 6 7

4 8 6

5 1 9

6 4 7

7 2 5

8 1 4

9 4 7

10 6 2

Query Points:

ID X Y

1 4 7

- Algorithm: Best-First Search using R-tree
- The R-tree was constructed to index the dataset. A priority queue was used to explore the most promising nodes (based on MINDIST) first. The algorithm efficiently reached a perfect match.

Output:

Nearest Neighbour: ID 6 or ID 9 $\rightarrow$ (4, 7)

- Both IDs 6 and 9 exactly match the query point. The algorithm correctly identifies the nearest neighbour with zero distance, confirming its accuracy.

7.2 Skyline Search

Dataset Used: sample_data_for_BBS_algorithm_based_skyline_search.txt

Sample Points:

ID X Y

1 5 10

2 8 7

3 12 5

4 3 12

5 10 8

6 6 6

7 9 4

8 2 14

9 13 3

10 15 2

11 4 9

12 11 6

13 7 3

14 1 13

15 14 1

Algorithm: Branch-and-Bound Skyline (BBS)

- Dataset was grouped, and bounding rectangles were created for each group.
- Representative points were evaluated for dominance.
- Only non-dominated groups were scanned.

Skyline**Output:**

Skyline Points: (1, 13), (2, 14), (3, 12), (4, 9), (5, 10)

- Skyline points are those not dominated by any other (i.e., no point has both a lower X and a higher Y). The BBS algorithm significantly reduced comparisons while preserving correctness.

8. Output Analysis**8.1 Nearest Neighbour Outputs**

Algorithm	Runtime (secs)	Scalable	Notes
Sequential NN	~3.80	Not scalable	Brute-force approach; inefficient for large-scale data
Best-First NN (R-tree)	~0.005	scalable	Fast due to spatial pruning using MINDIST
D&C Best-First NN	0.006	Highly scalable	Supports parallel search; best for large-scale inputs

8.1 Skyline Search Outputs

Algorithm	Runtime (secs)	Scalable	Notes
Sequential Skyline	~5.34	Not scalable	$O(n^2)$ comparisons; only viable for small inputs
Branch & Bound Skyline (BBS)	~0.18	scalable	MBR-based grouping improves speed significantly
Divide & Conquer BBS Skyline	0.23	Highly scalable	Parallel and optimized; best for large datasets

8.3 Summary Analysis

All algorithms produced correct outputs. However, runtime efficiency varied widely across implementations. Brute-force methods (Sequential NN and Skyline) were significantly slower. Best-First Search and BBS methods offered substantial performance gains, especially on structured spatial data.

Divide-and-Conquer strategies consistently showed the best scalability, particularly beneficial for larger datasets due to their ability to partition, parallelize, and reduce computational load.

9. Conclusion

This project successfully implemented and analyzed two critical spatial search problems: **Nearest Neighbour Search** and **Skyline Search**, using both baseline and optimized algorithmic approaches. Across both tasks, correctness was maintained in all implementations, with clear improvements in performance and scalability observed through the use of advanced techniques.

For Nearest Neighbour Search, transitioning from a brute-force sequential method to **Best-First Search with R-tree indexing** led to significant reductions in runtime. The **Divide-and-Conquer extension** further improved scalability, making it suitable for large spatial datasets where speed and efficiency are critical.

In Skyline Search, the **Branch-and-Bound Skyline (BBS)** approach effectively reduced computation through group-level pruning using representative points. The **Divide-and-Conquer BBS** strategy demonstrated superior scalability, efficiently handling large datasets with minimal additional overhead.

The case studies confirmed algorithm correctness, while performance metrics clearly showed the limitations of sequential methods and the advantages of structured indexing and decomposition strategies. This project highlighted the importance of **spatial indexing**, **search space reduction**, and **algorithmic design** in developing high-performance spatial data processing solutions.

Overall, the experience provided practical insights into the trade-offs between algorithm complexity and performance, and reinforced the importance of designing **scalable, efficient solutions** for real-world spatial applications.

Video Presentation Link

[Here](#)